Autoencoders: Use Cases, Variants, and Architectural Modifications

Research assignment report for an AI Advanced Course lecture task

Prepared for	AI Advanced Course
Document type	Structured research brief
Focus	Use cases, types, and modular architectural changes
Date	9 April 2026

What this report argues

Autoencoders are best understood as a flexible encoder-decoder framework. In most applications, we do not abandon that core design; instead, we change the bottleneck, training objective, regularization, or backbone so the same basic idea fits a new problem.

1. Introduction and assignment framing

An autoencoder is a neural network that learns to compress an input into a latent representation and then reconstruct it. The central idea is not simple copying; instead, the model is forced to learn a useful internal representation by passing information through a bottleneck or by adding constraints to the latent space or training objective.1

This report answers the three questions implied by the lecture task: **where autoencoders are used in practice, what the major variants are, and whether new tasks require a completely new architecture or just modular changes to an existing one.** The short answer is that autoencoders are a flexible framework, and most application-specific versions keep the same encoder-latent-decoder skeleton while changing the loss, bottleneck, or input pipeline. 2

2. Core idea: what an autoencoder actually learns

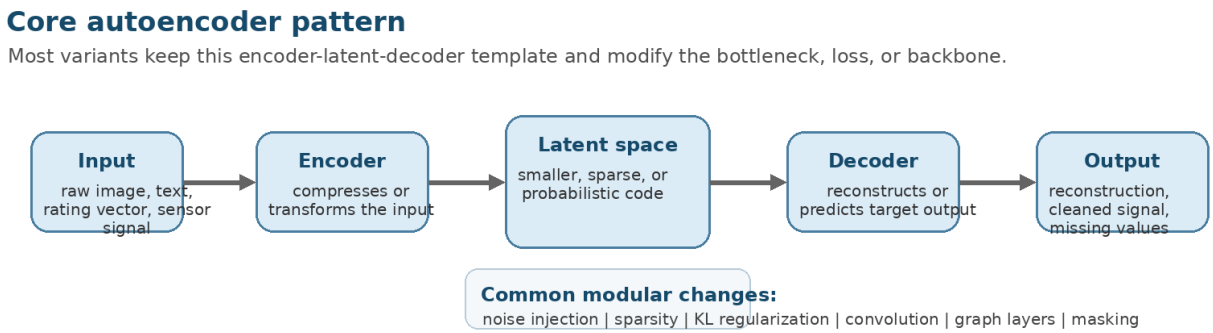

Figure 1. The general autoencoder template. Most variants keep this structure and modify one or two pieces instead of replacing the whole design.

In the simplest case, an autoencoder takes input x, maps it to a hidden code z, and reconstructs an output x-hat. If the model is undercomplete, sparse, noisy during training, or probabilistic in latent space, it cannot rely on trivial copying. The pressure created by those constraints is what makes the learned representation useful for downstream work such as anomaly scoring, feature extraction, generation, or missing-value estimation. [[FN01]]

3. Real-world use cases of autoencoders

3.1 Image denoising, restoration, and representation learning

One of the earliest practical uses of autoencoders is denoising. A denoising autoencoder receives a corrupted version of an input and learns to reconstruct the clean version. This changes the learning

1G. E. Hinton and R. R. Salakhutdinov, 'Reducing the Dimensionality of Data with Neural Networks,' Science 313(5786), 504-507, 2006.

2F. Daneshfar, K. Berahmand, E. S. Salehi, Y. Li, and Y. Xu, 'Autoencoders and their applications in machine learning: a survey,' Artificial Intelligence Review 57:28, 2024.

problem from simple copying to robust feature learning, which is why the same idea later became useful in image restoration, speech enhancement, and more general representation learning.[3]

- **Practical picture:** a medical image or industrial camera frame may contain noise, blur, or sensor artifacts. The autoencoder learns a clean latent summary and reconstructs a better version for later use.
- **Why it works:** the network is rewarded for preserving structure while discarding corruption.

3.2 Anomaly detection

Autoencoders are widely used for anomaly detection because a model trained mainly on normal data tends to reconstruct normal examples better than unusual ones. This makes reconstruction error a useful anomaly score in domains such as cybersecurity, predictive maintenance, fraud screening, and medical quality control.[4]

- **Practical picture:** a factory trains an autoencoder on normal vibration and temperature readings from healthy machines. If a new pattern reconstructs poorly, the system flags it for inspection before failure.
- **Key caveat:** good anomaly detection depends heavily on what the model saw during training and on how the threshold is calibrated.

3.3 Recommendation systems

In recommendation, the input is often a partially observed user-item vector. Autoencoder-based recommender models reconstruct that vector so the missing entries become predicted preferences. AutoRec is a classic example, showing that an autoencoder can learn nonlinear latent representations for collaborative filtering and perform strongly on MovieLens and Netflix-style datasets.[5]

- **Practical picture:** if a user has rated only a small subset of movies, the model uses that partial vector to infer which unseen movies are likely to match the user.

3.4 Graph and network analysis

When the data are relational instead of grid-like, graph autoencoders adapt the encoder to graph structure. Variational Graph Auto-Encoders learn node embeddings and then decode likely relationships, which makes them useful for link prediction and graph representation learning.[6]

- **Practical picture:** in a citation or social graph, the model can estimate which new connections are likely to appear, helping with recommendation, graph completion, or fraud analysis.

3P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, 'Extracting and Composing Robust Features with Denoising Autoencoders,' ICML, 2008.

4Daneshfar et al. survey a broad application range for autoencoders, including anomaly detection and fault diagnosis: Artificial Intelligence Review 57:28, 2024.

5S. Sedhain, A. K. Menon, S. Sanner, and L. Xie, 'AutoRec: Autoencoders Meet Collaborative Filtering,' WWW Companion, 2015.

6T. N. Kipf and M. Welling, 'Variational Graph Auto-Encoders,' arXiv:1611.07308, 2016.

3.5 Missing-data imputation

Autoencoders are also useful when tabular records have missing fields. Surveys of the imputation literature show that standard, denoising, and variational autoencoders can learn structure from incomplete rows and generate plausible replacements for missing values, particularly when relationships among variables are nonlinear.7

- **Practical picture:** a healthcare dataset may have missing lab values or demographic fields. The autoencoder learns correlations among columns and fills in estimates before a downstream predictor is trained.

3.6 Self-supervised pretraining with masked autoencoders

Masked autoencoders show how the same family of ideas now powers modern self-supervised learning. Instead of reconstructing the full input directly, the model sees only part of an image and reconstructs the missing patches. This allows the encoder to learn features that transfer well to downstream vision tasks.8

- **Practical picture:** large image models can be pretrained without labels, then fine-tuned for classification, detection, or segmentation.

3.7 Emerging use: sparse autoencoders for LLM interpretability

A newer application appears in mechanistic interpretability. Sparse autoencoders can be trained on the internal activations of a larger language model so that a sparse bottleneck separates many superposed concepts into cleaner features. This is not the traditional reconstruction use case, but it still follows the same core logic: compress activations into a more useful internal basis and decode them back.9

*Table 1A. Use-case matrix by industry and problem type*10

Industry or domain	Typical data	Practical problem solved	Useful AE pattern	How it is applied in practice
Manufacturing and industrial IoT	sensor streams, vibration, temperature	fault detection and early warning	undercomplete or denoising AE	Train on normal machine behavior, then flag unusually high reconstruction error for new readings.

7R. C. Pereira, M. S. Santos, P. P. Rodrigues, and P. H. Abreu, 'Reviewing Autoencoders for Missing Data Imputation: Technical Trends, Applications and Outcomes,' Journal of Artificial Intelligence Research 69, 1255-1285, 2020.

8K. He, X. Chen, S. Xie, Y. Li, P. Dollar, and R. Girshick, 'Masked Autoencoders Are Scalable Vision Learners,' CVPR, 2022.

9L. Gao et al., 'Scaling and Evaluating Sparse Autoencoders,' arXiv:2406.04093, 2024.

10This matrix is a synthesis of the cited literature, especially Daneshfar et al. (survey), Vincent et al. (denoising), Sedhain et al. (recommendation), Kipf and Welling (graph), Pereira et al. (imputation), He et al. (masked AE), and Gao et al. (sparse AE interpretability).

Medical imaging	MRI, CT, ultrasound, retinal images	noise removal, artifact reduction, feature learning	convolutional or denoising AE	Corrupt or noisy scans are mapped back to cleaner reconstructions before a radiologist or downstream model uses them.
Cybersecurity	network traffic features, log data	intrusion and anomaly detection	AE with anomaly scoring	Learn the normal pattern of traffic; rare attack patterns reconstruct poorly and become alerts.
E-commerce and media	user-item interactions, ratings, clicks	recommendation and missing preference prediction	AutoRec-style AE	Observed user or item vectors are reconstructed so missing entries become predicted preferences.

Table 1B. Use-case matrix by industry and problem type (continued)

Industry or domain	Typical data	Practical problem solved	Useful AE pattern	How it is applied in practice
Healthcare and business analytics	tabular records with missing fields	data imputation before modeling	denoising or variational AE	Partially observed rows are encoded and decoded to estimate plausible missing values.
Graphs and networks	citation graphs, social networks, knowledge graphs	link prediction and node embeddings	graph AE or VGAE	Graph encoders learn latent node representations and the decoder estimates which edges are likely to exist.
Computer vision pretraining	image patches	self-supervised representation learning	masked AE	Large parts of an image are masked, and the model learns reusable features by reconstructing the missing patches.
LLM interpretability (emerging)	internal model activations	recovering sparse, interpretable features	sparse AE	Activation vectors from a larger model are reconstructed through a sparse bottleneck to separate meaningful features.

4. Major types of autoencoders

The main variants can be understood as targeted edits to the basic encoder-decoder design. Each variant changes one important ingredient: the bottleneck, the training signal, the regularizer, the latent distribution, or the layer type used to process the data. [[FN02]]

Table 2. Major autoencoder variants and the main modular change they introduce

Variant	Main change	Why it helps	Typical fit
Undercomplete AE	small bottleneck	forces compression	dimensionality reduction
Sparse AE	sparsity constraint or top-k activity	selective features	feature discovery, interpretability
Denoising AE	corrupt input during training	robust reconstruction	restoration, representation learning
Contractive AE	penalize encoder sensitivity	local invariance	stable features on noisy data
Variational AE	probabilistic latent distribution with KL term	sampling and generative modeling	generation, uncertainty, anomaly variants

Table 2 (continued). Major autoencoder variants and the main modular change they introduce

Variant	Main change	Why it helps	Typical fit
Adversarial AE	match latent space to a prior adversarially	structured latent space	generative or semi-supervised tasks
Convolutional AE	convolutional encoder and decoder	preserve spatial structure	images and video
Sequence AE	temporal or recurrent blocks	retain order over time	speech and time series
Graph AE / VGAE	graph-aware encoder	relational structure	links, embeddings, graph mining
Masked AE	reconstruct masked parts only	efficient self-supervision	vision pretraining

4.1 Undercomplete and sparse autoencoders

An undercomplete autoencoder uses a latent space smaller than the input, so compression itself becomes the learning pressure. A sparse autoencoder can keep the latent space large but forces only

a few hidden units to activate. In both cases, the network is pushed toward selective feature learning rather than memorization. [[FN01]]11

4.2 Denoising and contractive autoencoders

Denoising autoencoders inject corruption into the input and reconstruct the clean target, while contractive autoencoders add a penalty that makes the encoder locally insensitive to small perturbations. Both changes aim for robustness, but they do so in different ways: one through the training data, the other through the objective function. [[FN03]]12

4.3 Variational and adversarial autoencoders

Variational autoencoders replace a point latent code with a probability distribution and optimize a reconstruction term together with a KL-divergence term. Adversarial autoencoders also try to shape the latent space, but they use an adversarial objective to match the aggregated posterior to a chosen prior. These variants are especially useful when sampling, generative modeling, or uncertainty-aware reasoning matters.1314

4.4 Convolutional, sequence, graph, and masked autoencoders

Some variants are driven less by a new loss and more by the structure of the data. Convolutional autoencoders preserve spatial locality for images. Sequence autoencoders use temporal blocks for speech or time series. Graph autoencoders use graph-aware message passing. Masked autoencoders change the self-supervision pattern by reconstructing only hidden content. In all these cases, the application changes the backbone, not the basic idea. [[FN02]] [[FN06]] [[FN07]]15

11A. Makhzani and B. Frey, 'k-Sparse Autoencoders,' arXiv:1312.5663, 2013.

12S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio, 'Contractive Auto-Encoders: Explicit Invariance During Feature Extraction,' ICML, 2011.

13A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey, 'Adversarial Autoencoders,' arXiv:1511.05644, 2015.

14D. P. Kingma and M. Welling, 'Auto-Encoding Variational Bayes,' arXiv:1312.6114, 2013.

15X. Lu, Y. Tsao, S. Matsuda, and C. Hori, 'Speech Enhancement Based on Deep Denoising Autoencoder,' Interspeech, 2013.

5. Do new use cases require a completely new architecture?

Most of the time, **no**. New use cases usually do not require discarding the autoencoder template. Instead, they require a sequence of modular decisions: What kind of latent code is needed? What should the loss encourage? What layers match the data modality? What signal should the model reconstruct? [[FN02]]

Table 3. The assignment question in one view: what changes when the use case changes?

What changes	Examples of modular edits	Why this is enough most of the time
Latent space	smaller bottleneck, sparse code, Gaussian latent distribution	The learning target remains a useful internal representation.
Objective function	reconstruction loss plus noise robustness, KL term, Jacobian penalty, adversarial regularization	The same network can learn a different behavior when optimized for a different goal.
Backbone layers	dense layers for tabular data, convolutions for images, graph convolutions for graphs, temporal blocks for sequences	The data structure changes, but the encoder-decoder idea stays intact.
Training signal	mask the input, corrupt the input, train only on normal data	These changes alter what the model learns without replacing the full architecture.
Decision rule at inference	reconstruction error threshold, latent sampling, edge scoring, missing-value fill-in	Many application differences appear in how the learned representation is used after training.

A helpful way to think about the design space is this: **the task determines the pressure you apply to the latent representation.** If you want robustness, add noise or sensitivity penalties. If you want generation, make the latent space probabilistic or structured. If the data are images, use convolutions. If the data are graphs, use a graph encoder. If you want anomaly detection, train mostly on normal examples and evaluate reconstruction error. [[FN03]] [[FN06]] [[FN11]]

There are cases where the architecture changes more substantially, especially when the data modality shifts from tabular vectors to images, sequences, or graphs. Even then, however, the change is usually still best described as a **backbone substitution inside the same encoder-decoder family** rather than as a completely unrelated new model. [[FN02]]

6. One-Paragraph Answer

Autoencoders are used in practice for denoising, anomaly detection, recommendation, graph learning, missing-data imputation, and self-supervised pretraining. The major variants - sparse, denoising, contractive, variational, adversarial, convolutional, sequence, graph, and masked autoencoders - are mostly created by modifying the bottleneck, loss function, regularization, or backbone. Therefore, different use cases usually do not require a

completely new architecture; they require modular changes that adapt the same encoder-decoder principle to a new data type or learning objective.

7. What this prepares you for in the next class

- **If the next session covers code, expect to see the same base training loop reused across variants.** The main differences will be how data are corrupted, how the latent layer is constrained, and what extra loss terms are added.
- **A practical coding pattern:** start with a vanilla undercomplete autoencoder, then add one change at a time: noise injection, sparsity, KL loss, convolutional layers, or graph layers.
- **A practical evaluation pattern:** measure not only reconstruction loss but also whether the learned representation actually helps the downstream task, such as anomaly detection quality or recommendation accuracy.

Q. In which Scenario does a Variational Autoencoder perform better than a Vanilla Autoencoder?

- A **Variational Autoencoder (VAE)** performs better than a **Vanilla Autoencoder** when you need the model to learn a **smooth, structured latent space** rather than only reconstruct training examples.
- **VAE is better in these scenarios**
- **1. Generating new realistic samples**
  If you want to create new images, faces, handwritten digits, or synthetic data, a VAE is usually better.
  Why: a vanilla autoencoder learns compressed representations, but its latent space is not organized well for sampling. A VAE forces the latent space to follow a probability distribution, so you can sample new points and decode them into meaningful outputs.
- **2. Interpolation between examples**
  If you want smooth transitions, like moving from one face to another or one digit style to another, a VAE works better.
  Why: nearby latent points in a VAE usually decode to similar, meaningful outputs.
- **3. Learning a more regularized representation**
  When you want latent features that are smoother, less overfit, and more interpretable for downstream tasks, a VAE is often preferable.
  Why: the KL-divergence term regularizes the latent space.
- **4. Handling uncertainty**
  If the task benefits from modeling uncertainty, a VAE is stronger.
  Why: it learns distributions for latent variables, not just single fixed encodings.
- **5. Missing data / imputation / probabilistic modeling**
  VAEs are often better when you need a probabilistic model of the data, such as filling missing values or learning data distributions.

- **Vanilla Autoencoder is better in these scenarios**
- A **Vanilla Autoencoder** is often better when:
  - your main goal is **accurate reconstruction**
  - you want **simple dimensionality reduction**
  - you want **feature compression**
  - you do **anomaly detection using reconstruction error**
  - you need a simpler model that is easier to train
- Why: vanilla autoencoders usually reconstruct more sharply because they are optimized mainly for reconstruction, while VAEs trade some reconstruction quality for a more structured latent space.

- **Easy rule to remember**
  - **Need better reconstruction?** → Vanilla Autoencoder
  - **Need generation, sampling, interpolation, or probabilistic latent space?** → VAE
- **One-line**
- A Variational Autoencoder performs better than a Vanilla Autoencoder when the goal is not just reconstruction but learning a smooth probabilistic latent space for tasks like generating new samples, interpolation, uncertainty modeling, and structured representation learning.

8. References

[1] Hinton, G. E., and Salakhutdinov, R. R. "Reducing the Dimensionality of Data with Neural Networks." Science 313(5786), 504-507, 2006. https://dbirman.github.io/learn/hierarchy/pdfs/Hinton2006.pdf

[2] Daneshfar, F., Berahmand, K., Salehi, E. S., Li, Y., and Xu, Y. "Autoencoders and their applications in machine learning: a survey." Artificial Intelligence Review 57:28, 2024. https://link.springer.com/content/pdf/10.1007/s10462-023-10662-6.pdf

[3] Vincent, P., Larochelle, H., Bengio, Y., and Manzagol, P.-A. "Extracting and Composing Robust Features with Denoising Autoencoders." ICML, 2008. https://www.cs.toronto.edu/~larocheh/publications/icml-2008-denoising-autoencoders.pdf

[4] Rifai, S., Vincent, P., Muller, X., Glorot, X., and Bengio, Y. "Contractive Auto-Encoders: Explicit Invariance During Feature Extraction." ICML, 2011. https://www-perso.iro.umontreal.ca/~vincentp/Publications/contractive_autoencoder_icml2011.pdf

[5] Sedhain, S., Menon, A. K., Sanner, S., and Xie, L. "AutoRec: Autoencoders Meet Collaborative Filtering." WWW Companion, 2015. https://users.cecs.anu.edu.au/~akmenon/papers/autorec/autorec-paper.pdf

[6] Kingma, D. P., and Welling, M. "Auto-Encoding Variational Bayes." arXiv:1312.6114, 2013. https://arxiv.org/abs/1312.6114

[7] Makhzani, A., Shlens, J., Jaitly, N., Goodfellow, I., and Frey, B. "Adversarial Autoencoders." arXiv:1511.05644, 2015. https://arxiv.org/abs/1511.05644

[8] Kipf, T. N., and Welling, M. "Variational Graph Auto-Encoders." arXiv:1611.07308, 2016. https://arxiv.org/abs/1611.07308

[9] He, K., Chen, X., Xie, S., Li, Y., Dollar, P., and Girshick, R. "Masked Autoencoders Are Scalable Vision Learners." CVPR, 2022. https://openaccess.thecvf.com/content/CVPR2022/papers/He_Masked_Autoencoders_Are_Scalable_Vision_Learners_CVPR_2022_paper.pdf

[10] Makhzani, A., and Frey, B. "k-Sparse Autoencoders." arXiv:1312.5663, 2013. https://arxiv.org/abs/1312.5663

[11] Pereira, R. C., Santos, M. S., Rodrigues, P. P., and Abreu, P. H. "Reviewing Autoencoders for Missing Data Imputation: Technical Trends, Applications and Outcomes." Journal of Artificial Intelligence Research 69, 1255-1285, 2020. https://www.jair.org/index.php/jair/article/view/12312

[12] Lu, X., Tsao, Y., Matsuda, S., and Hori, C. "Speech Enhancement Based on Deep Denoising Autoencoder." Interspeech, 2013. https://www.isca-archive.org/interspeech_2013/lu13_interspeech.pdf

[13] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., and Wu, J. "Scaling and Evaluating Sparse Autoencoders." arXiv:2406.04093, 2024. https://cdn.openai.com/papers/sparse-autoencoders.pdf